SRS — Módulo de Membresías:

1. Introducción

Este documento especifica los requerimientos del Módulo de Membresías bajo la norma ISO/IEC/IEEE 29148:2018, y sirve de base para el diseño (UML, fichas IEEE 1016) y el backlog ágil.

1.1 Propósito

Definir de forma verificable qué debe hacer el módulo y con qué calidad, para que el equipo de desarrollo, el Product Owner y quien evalúe el proyecto compartan una sola fuente de verdad. Destinatarios: analista, desarrolladores, QA, docente.

1.2 Alcance

El producto se llama **API de Modulos de Membresías** y expone servicios REST para tres capacidades:

- **Catálogo de planes:** administrar y consultar los planes Básico, Pro y Elite.
- **Gestión de membresías:** registrar, consultar y cancelar membresías, y calcular su estado (Activo, Por vencer, Vencido).
- **Procesamiento de pagos:** cobrar la membresía con tarjeta mediante la pasarela externa y guardar cada transacción.

Fuera de alcance: autenticación y emisión del JWT, ficha completa del cliente y generación de reportes. Son módulos externos con los que esta API solo se integra.

1.3 Definiciones y acrónimos

Término	Definición
API	Interfaz de programación de aplicaciones; aquí, servicios REST sobre HTTPS con cuerpo JSON
JWT	JSON Web Token; credencial firmada emitida por el Módulo de Autenticación

Plan	Oferta comercial con nombre, precio, duración y beneficios: Básico, Pro o Elite
Membresía	Contrato vigente de un cliente con un plan, con fecha de inicio y fecha de fin
Estado visual	Activo, Por vencer (7 días o menos para fecha_fin) o Vencido; se calcula, no se almacena
Pasarela de Pago	Sistema externo que autoriza o rechaza el cobro con tarjeta
SRS	Software Requirements Specification, este documento
REQ-FN / REQ-NF	Identificadores de requerimiento funcional / no funcional
HU / RN	Historia de usuario / regla de negocio del backlog

2. Descripción general

La API de Membresías es un servicio independiente, consumido por la app móvil del gimnasio, que se apoya en cuatro sistemas externos y en su propia base de datos.

2.1 Perspectiva del producto

La API se desarrolla como módulo autónomo (monolito modular). Persiste planes, membresías y pagos en SQL Server y guarda el catálogo de planes en caché Redis (patrón Cache-Aside). Depende de:

- **Módulo de Autenticación:** emite y valida el JWT.
- **Módulo de Clientes:** confirma que el cliente existe y está Activo.
- **Pasarela de Pago:** autoriza o rechaza el cobro.
- **Módulo de Reportes:** recibe datos de ingresos y vencimientos.

2.2 Funciones principales

1. Consultar, crear, editar y desactivar planes (Básico, Pro, Elite).
2. Registrar, consultar y cancelar membresías.
3. Calcular el estado visual (Activo, Por vencer, Vencido) de cada membresía.
4. Procesar pagos con tarjeta y guardar cada intento (aprobado o rechazado).
5. Proteger todos los servicios con JWT y validar al cliente antes de vender.
6. Enviar datos de ingresos y vencimientos al Módulo de Reportes.

2.3 Clases de usuario

Rol	Qué hace en el módulo	Servicios que usa
Cliente	Consulta planes y el estado de su membresía	Catálogo, estado de membresía
Recepcionista	Registra y cancela membresías y cobra	Membresías, pagos
Administrador	Gestiona planes y consulta el historial de pagos	Planes, historial de pagos
Módulos externos	Autenticación, Clientes, Pasarela, Reportes	Interfaces de la sección 5

2.4 Restricciones

- El catálogo contiene tres planes (Básico, Pro y Elite) y solo los planes activos pueden venderse (RN-04).
- Base de datos SQL Server; caché Redis; contrato REST documentado en OpenAPI 3.0.3.
- Formato de error único: `{ success: false, error: { code, message } }`.
- Moneda por defecto: GTQ; el monto de un pago es mayor a 0.

2.5 Supuestos y dependencias

- El Módulo de Autenticación está disponible y entrega JWT con el rol del usuario.
- El Módulo de Clientes responde de forma consultable por identificador de cliente.
- La pasarela de pago es un servicio externo; sus fallas no deben corromper el estado de una membresía.
- Si un supuesto cambia (por ejemplo, otro proveedor de pagos), cambian los requerimientos de la sección 5.

3. Requerimientos funcionales

ID	Requerimiento	Origen	Prioridad
REQ-FN-001	El sistema debe devolver el nombre, precio, duración y beneficios de los planes con activo = 1 cuando reciba `GET /planes`.	HU-01, RN-04	Must
REQ-FN-002	El sistema debería crear un plan cuando el Administrador envíe `POST /planes` con nombre Básico, Pro o Elite que aún no exista.	HU-02	Should
REQ-FN-003	El sistema debería actualizar el precio y los beneficios de un plan cuando el Administrador envíe `PUT /planes/{id}` y el plan exista.	HU-02	Should
REQ-FN-004	El sistema debería rechazar con 400 la creación o edición de un plan cuando el precio sea menor o igual a 0 o el nombre no sea Básico, Pro o Elite.	HU-02	Should
REQ-FN-005	El sistema podrá desactivar un plan (activo = 0) sin eliminarlo cuando el Administrador envíe `PATCH /planes/{id}`, conservando las membresías que ya lo usan.	HU-03, RN-04	Could

REQ-FN-006	El sistema debería invalidar el catálogo almacenado en caché cuando se cree, edite o desactive un plan.	HU-02, HU-03	Should

3.2 Gestión de membresías (EPIC-02)

ID	Requerimiento	Origen	Prioridad
REQ-FN-007	El sistema debe registrar una membresía con estado Activa cuando reciba `POST /membresias` con un cliente válido, un plan activo y fechas válidas.	HU-04	Must
REQ-FN-008	El sistema debe rechazar con 400 el registro de una membresía cuando fecha_fin no sea posterior a fecha_inicio.	RN-03	Must
REQ-FN-009	El sistema debe rechazar el registro de una membresía con 404 cuando el plan no exista y con 422 cuando el plan esté inactivo.	RN-04	Must
REQ-FN-010	El sistema debe devolver las membresías de un cliente, de la más reciente a la más antigua, cuando reciba `GET /membresias` con id_cliente.	HU-05	Must
REQ-FN-011	El sistema debe devolver los datos de una membresía, incluido su estado visual, cuando reciba `GET /membresias/{id}`, y responder 404 cuando no exista.	HU-06	Must
REQ-FN-012	El sistema debe calcular el estado visual en cada consulta: Vencido si la membresía está cancelada o su fecha_fin ya pasó, Por vencer si faltan 7 días o menos, y Activo en cualquier otro caso.	HU-06, HU-07, RN-05	Must

REQ-FN-013	El sistema podrá cancelar una membresía cuando el Recepcionista envíe `PATCH /membresias/{id}` con estado Cancelada.	HU-08	Could
REQ-FN-014	El sistema podrá rechazar con 409 la cancelación de una membresía cuando ya esté Cancelada o Vencida.	HU-08	Could

3.3 Procesamiento de pagos

ID	Requerimiento	Origen	Prioridad
REQ-FN-015	El sistema debe procesar el pago de una membresía cuando reciba `POST /payments/process` con membresía existente, monto y moneda válidos.	HU-09, RN-06, RN-07	Must
REQ-FN-016	El sistema debe rechazar con 400 un pago cuando el monto sea menor o igual a 0 o la moneda no sea un código ISO 4217.	RN-06	Must
REQ-FN-017	El sistema debe rechazar con 404 un pago cuando la membresía indicada no exista.	RN-07	Must
REQ-FN-018	El sistema debe enviar la solicitud de cobro a la Pasarela de Pago y responder con la decisión (APPROVED o DECLINED) y el transaction_id cuando el pago sea válido.	HU-09	Must
REQ-FN-019	El sistema debe registrar cada intento de pago, aprobado o rechazado, con monto, moneda, estado, transaction_id y fecha.	HU-10	Must

REQ-FN-020	El sistema debe asignar a cada pago registrado un transaction_id único.	HU-10	Must
REQ-FN-021	El sistema debe responder 504 y no registrar el pago como APPROVED cuando la Pasarela de Pago no responda dentro del tiempo límite.	HU-09	Must
REQ-FN-022	El sistema debería devolver los pagos de una membresía cuando el Administrador envíe `GET /membresias/{id}/pagos`.	HU-11	Should

3.4 Integración con otros módulos

ID	Requerimiento	Origen	Prioridad
REQ-FN-023	El sistema debe validar la firma y la vigencia del JWT cuando reciba una petición a un endpoint protegido.	HU-12, RN-01	Must
REQ-FN-024	El sistema debe responder 401 cuando falte el encabezado `Authorization: Bearer <token>` o el token sea inválido o esté vencido.	RN-01	Must
REQ-FN-025	El sistema debe responder 403 cuando el rol contenido en el token no tenga permiso sobre el endpoint solicitado.	HU-12, HU-02, HU-11	Must
REQ-FN-026	El sistema debe consultar al Módulo de Clientes si el cliente existe y está Activo antes de registrar una membresía.	HU-13, RN-02	Must

REQ-FN-027	El sistema debe rechazar el registro de una membresía con 404 cuando el cliente no exista y con 422 cuando el cliente no esté Activo.	HU-13, RN-02	Must
REQ-FN-028	El sistema debe responder 503 y no crear la membresía cuando el Módulo de Clientes no esté disponible.	HU-13	Must
REQ-FN-029	El sistema debería enviar al Módulo de Reportes los datos de ingresos y vencimientos cuando registre un pago APPROVED.	HU-14	Should
REQ-FN-030	El sistema debería completar el pago y dejar el envío pendiente de reintento cuando el Módulo de Reportes no responda.	HU-14	Should

3.5 Formato de respuestas

ID	Requerimiento	Origen	Prioridad
REQ-FN-031	El sistema debe responder todo error con el formato `{ success: false, error: { code, message } }`.	RN-08	Must
REQ-FN-032	El sistema debe responder toda consulta exitosa con el formato `{ success: true, data }`.	RN-08	Must

4. Requerimientos no funcionales

4.1 Seguridad

ID	Requerimiento	Verificación
REQ-NF-001	El sistema debe aceptar peticiones únicamente por HTTPS con TLS 1.2 o superior.	Una petición HTTP sin cifrado es rechazada o redirigida
REQ-NF-002	El sistema debe rechazar con 401 todo JWT con firma alterada o con algoritmo `none`.	Prueba con un token manipulado devuelve 401
REQ-NF-003	El sistema no debe almacenar número de tarjeta ni CVV; solo debe guardar el transaction_id de la pasarela.	Revisión de `schema.sql`: ninguna columna con datos de tarjeta
REQ-NF-004	El sistema debe ejecutar todo acceso a SQL Server mediante consultas parametrizadas.	Revisión de código y prueba de inyección SQL sin efecto
REQ-NF-005	El sistema no debe incluir trazas de pila ni sentencias SQL en el mensaje de error que devuelve al cliente.	Provocar un error interno y revisar el cuerpo de la respuesta

4.2 Rendimiento

ID	Requerimiento	Verificación
REQ-NF-006	El sistema debe responder `GET /planes` en 200 ms o menos (percentil 95) con 100 usuarios concurrentes cuando el catálogo esté en caché.	Prueba de carga
REQ-NF-007	El sistema debe responder las consultas de membresías en 500 ms o menos (percentil 95) con 100 usuarios concurrentes.	Prueba de carga
REQ-NF-008	El sistema debe esperar la respuesta de la Pasarela de Pago un máximo de 10 segundos antes de responder 504.	Simular una pasarela lenta y medir el tiempo de respuesta
REQ-NF-009	El sistema debe expirar el catálogo en caché a más tardar 10 minutos después de cargarlo.	Consultar el TTL de la clave en Redis

4.3 Disponibilidad y tolerancia a fallos

ID	Requerimiento	Verificación
REQ-NF-010	El sistema debe estar disponible al menos 99 % del tiempo en cada mes.	Monitoreo mensual de disponibilidad
REQ-NF-011	El sistema debe seguir respondiendo `GET /planes` desde SQL Server cuando Redis no esté disponible.	Apagar Redis y verificar que la respuesta sigue siendo 200

4.4 Usabilidad e interoperabilidad

ID	Requerimiento	Verificación
REQ-NF-012	El sistema debe publicar un contrato OpenAPI 3.0.3 que valide sin errores y describa todos los endpoints implementados.	Validar el YAML en Swagger Editor
REQ-NF-013	El sistema debe devolver en cada error un `code` constante y un `message` en español que indique qué dato corregir.	Revisión de los mensajes de error en pruebas de API

4.5 Mantenibilidad e integridad de datos

ID	Requerimiento	Verificación
REQ-NF-014	El sistema debe separar el código en capas Controllers, Services y Repositories, sin acceso directo de los Controllers a la base de datos.	Revisión del diagrama de paquetes y de las dependencias del código
REQ-NF-015	El sistema debe contar con pruebas unitarias que cubran al menos 70 % de las líneas de la capa Services.	Reporte de cobertura
REQ-NF-016	El sistema debe imponer en la base de datos las claves foráneas y las restricciones fecha_fin > fecha_inicio y monto > 0.	Intentar insertar datos que violen cada restricción y verificar el rechazo